Credit Default Risk — Technical Write-up

Credit Default Risk — Technical Write-up

Vitto DS Intern Assessment

Approach

The task was framed as an end-to-end credit risk pipeline: load and profile a real dataset, engineer domain-relevant features, train and compare classifiers, and translate findings into operational lending decisions. The overarching principle was to treat this as if preparing work for a credit risk team reviewing it the next morning — not as an academic exercise.

EDA first, modelling second. Before touching a model, I spent time understanding the data distribution, class balance, and the known data quality issues in the UCI dataset. Notably: - `EDUCATION` and `MARRIAGE` both contain undocumented values (0, 5, 6 and 0 respectively), handled by merging them into the existing “Other” category. - `PAY_x` uses two distinct negative codes (-1 = paid duly, -2 = no consumption) that are semantically different but both non-delinquent. - Several clients have negative `BILL_AMT` values (credit balances from overpayment), which produce nonsensical ratios if used directly as denominators.

All cleaning decisions were documented inline with explicit rationale.

Feature Engineering Rationale

Three engineered features were created to summarise repayment behaviour over six months:

- **`TOTAL_DELAY_MONTHS`** — the number of months with any recorded payment delay. This proved to be the single strongest predictor (XGBoost importance: 0.318), outperforming any individual `PAY_x` column. This makes economic sense: cumulative delinquency is a better signal of structural financial distress than any single month’s behaviour.
- **`AVG_UTIL_RATE`** — average credit utilisation. Negative bill amounts were clipped to zero before computing ratios to avoid sign-inverted values.
- **`AVG_PAY_RATIO`** — average fraction of each bill repaid. Clients with no bills across all six months were imputed with 1.0 (no outstanding balance = not a defaulter).

Modelling Choices and Trade-offs

Why Logistic Regression as baseline?
It is fast, interpretable, and provides coefficients that directly reflect each feature's marginal contribution. It also makes assumptions (linearity, no interactions) that are clearly violated here — making the gap with XGBoost informative rather than incidental.

Why XGBoost over Random Forest?
XGBoost handles mixed numeric/one-hot features well, supports `scale_pos_weight` natively for class imbalance, trains faster on this dataset size, and produces gain-based feature importances that are more discriminative than Random Forest's impurity-based scores.

Why `class_weight='balanced'` over SMOTE?
SMOTE must be applied inside each CV fold (via a `Pipeline`) to avoid data leakage — synthetic minority samples in the training fold are near-identical to held-out validation samples. This complexity adds risk without a material performance gain for a dataset of this size and imbalance ratio. `class_weight='balanced'` achieves the same upweighting of minority errors with no leakage risk and is simpler to audit.

Validation design:
An 80/20 stratified train/test split was performed first. Stratified 5-fold CV was run only on the training set. Final metrics (ROC curve, confusion matrix) were computed exclusively on the held-out test set. This design ensures no optimistic bias from CV leaking into reported numbers.

Results

Model	AUC-ROC	Recall	F1	CV AUC
Logistic Regression	0.745	0.563	0.512	0.758 ± 0.008
XGBoost	**0.778**	**0.625**	**0.537**	**0.782 ± 0.008**

The XGBoost model correctly flags **62% of clients who will default** (recall), with 47% precision — meaning roughly half of flagged clients do in fact default. For a lending team, this represents a practical early-warning signal: a targeted call list where every other conversation is with a genuine at-risk borrower.

The near-identical CV and test AUC scores (0.782 vs 0.778) confirm the model is not overfitting.

What I Would Improve Given More Time

1. **Hyperparameter tuning** — A `RandomizedSearchCV` over XGBoost's `max_depth`, `learning_rate`, `subsample`, and `colsample_bytree` could push AUC-ROC above 0.80. I used sensible defaults here to keep the notebook readable.
2. **SHAP explanations** — Global `beeswarm` plots and per-client `waterfall` plots would make the model's reasoning auditable at the individual account level — important for regulatory compliance in a live lending product.

3. **Fairness analysis** — Comparing False Positive Rates across demographic groups (sex, education) would reveal whether the model systematically over-flags certain groups. Given that high school-educated borrowers have the highest default rate (25%) while Other/Undocumented are lowest (7%), a fairness-aware threshold calibration may be warranted.
4. **Temporal validation** — The dataset covers Apr–Sep 2005. A walk-forward validation (train on Apr–Jul, test on Aug–Sep) would better simulate real-world deployment where the model is trained on historical data and applied to future clients.
5. **Probability calibration** — Logistic Regression probabilities are well-calibrated by default, but XGBoost scores may require Platt scaling or isotonic regression before using them as literal default probability estimates.

This assessment is confidential and submitted for evaluation by Vitto only.